Experiment No. 1(a) Date:

Experiment Question

Implement and demonstrate the FIND-S algorithm for finding the most specific hypothesis based on a given set of training data samples. [Sub-program (a): EnjoySport data set, using pandas]

Aim

To implement the FIND-S algorithm in Python using pandas and to find the most specific hypothesis consistent with all the positive training examples of the EnjoySport data set.

Concept

FIND-S is a concept learning algorithm. It starts with the most specific hypothesis in the hypothesis space H and generalises it only as much as is needed to cover each positive training example. Negative examples are ignored.

A hypothesis is a vector of attribute constraints. Each constraint is a specific value, "?" (any value is acceptable) or "0" (no value is acceptable). The output is the maximally specific hypothesis that is consistent with the positive examples, provided the data is free of errors and the target concept is contained in H.

Dataset Description

The EnjoySport data set (enjoysport.csv) has 4 training examples, 6 input attributes (Sky, Temp, Humidity, Wind, Water, Forecast) and the target attribute EnjoySport (Yes / No).

Sky	Temp	Humidity	Wind	Water	Forecast	EnjoySport
Sunny	Warm	Normal	Strong	Warm	Same	Yes
Sunny	Warm	High	Strong	Warm	Same	Yes
Rainy	Cold	High	Strong	Warm	Change	No
Sunny	Warm	High	Strong	Cool	Change	Yes

Algorithm

1. Initialise h to the most specific hypothesis in H: <0, 0, ..., 0>.
2. For each training example x:
   - If x is a negative example, ignore it.
   - If x is a positive example, then for each attribute constraint a(i) in h: if a(i) is satisfied by x, do nothing; otherwise replace a(i) by the next more general constraint that is satisfied by x ("?").
3. Output the hypothesis h.

Step-by-step Procedure

1. Import the pandas library and store the training data in enjoysport.csv.
2. Read the CSV file with read_csv() and separate the attributes (all columns except the last) from the target (last column).

3. Initialise the hypothesis with "0" for every attribute.
4. Take each example one by one. For a positive example, copy it into h if h is still all "0"; otherwise replace every attribute of h that differs from the example by "?".
5. Skip the negative examples.
6. Print the hypothesis after every example and display the final hypothesis.

Formulae

h = < a1, a2, ..., an >, ai ∈ { specific value, ?, 0 }

Initial hypothesis: h0 = < 0, 0, ..., 0 >

For a positive example x: ai = ai if ai = xi ; ai = ? otherwise

h satisfies x ⇔ for every i, ai = ? or ai = xi

Python Program

ex-1a.py - D:/ex-1a.py (3.10.0)

File Edit Format Run Options Window Help

```
# FIND-S Algorithm - EnjoySport data (using pandas)
import pandas as pd

data = pd.read_csv("enjoysport.csv")

print("FIND-S ALGORITHM")
print("----------------")
print(data)
print()

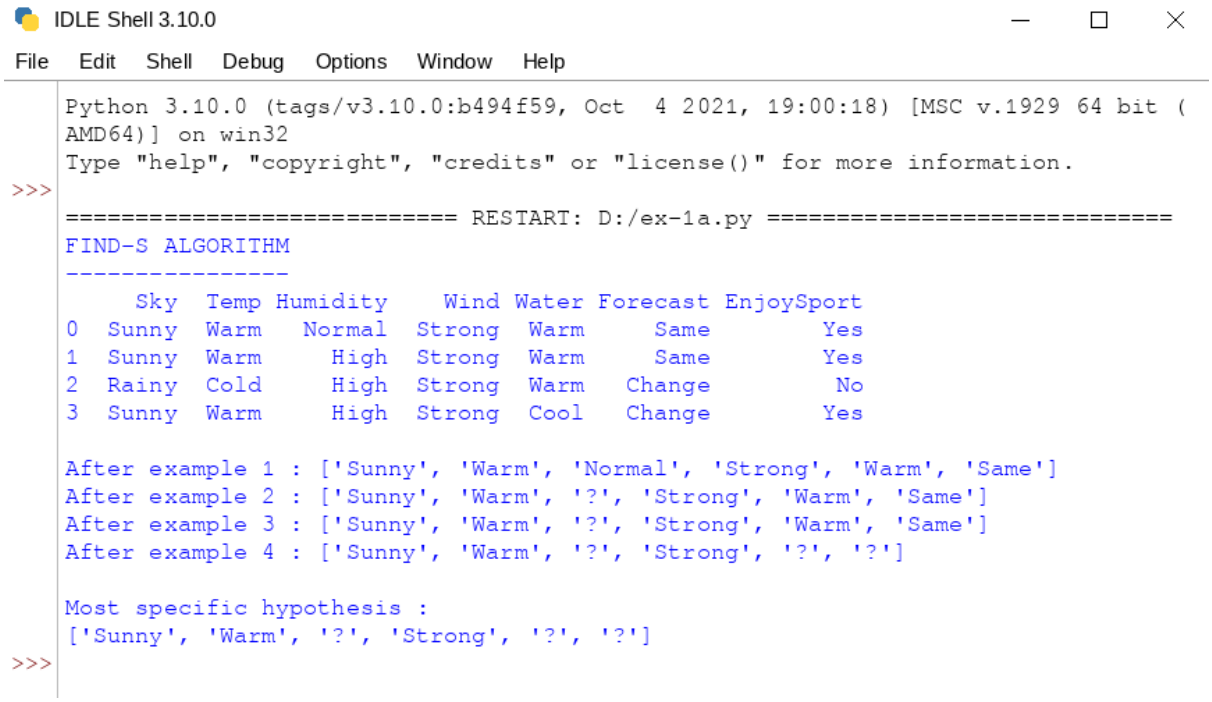
attributes = data.iloc[:, :-1].values
target = data.iloc[:, -1].values

# Step 1 : start with the most specific hypothesis
hypothesis = ["0"] * attributes.shape[1]

# Step 2 : consider only the positive examples
for i in range(len(target)):
    if target[i] == "Yes":
        if hypothesis[0] == "0":
            hypothesis = list(attributes[i])
        else:
            for j in range(len(hypothesis)):
                if hypothesis[j] != attributes[i][j]:
                    hypothesis[j] = "?"
    print("After example", i + 1, ":", hypothesis)

print()
print("Most specific hypothesis :")
print(hypothesis)
```

Output



Result / Conclusion

The FIND-S algorithm was implemented successfully on the EnjoySport data set. The most specific hypothesis consistent with the positive examples is <Sunny, Warm, ?, Strong, ?, ?>, i.e. the person enjoys the sport when the sky is sunny, the air is warm and the wind is strong.

Experiment No. 1(b) Date:

Experiment Question

Implement and demonstrate the FIND-S algorithm for finding the most specific hypothesis based on a given set of training data samples. [Sub-program (b): Weather data set, using the csv module]

Aim

To implement the FIND-S algorithm in Python using the csv module and to find the most specific hypothesis consistent with all the positive training examples of the Weather data set.

Concept

FIND-S is a concept learning algorithm. It starts with the most specific hypothesis in the hypothesis space H and generalises it only as much as is needed to cover each positive training example. Negative examples are ignored.

A hypothesis is a vector of attribute constraints. Each constraint is a specific value, "?" (any value is acceptable) or "0" (no value is acceptable). The output is the maximally specific hypothesis that is consistent with the positive examples, provided the data is free of errors and the target concept is contained in H.

Dataset Description

The Weather data set (weather.csv) has 5 training examples, 4 input attributes (Outlook, Temp, Humidity, Wind) and the target attribute Play (Yes / No) which tells whether the outdoor game can be played.

Outlook	Temp	Humidity	Wind	Play
Sunny	Warm	Normal	Weak	Yes
Sunny	Warm	High	Weak	Yes
Rainy	Cold	High	Strong	No
Sunny	Hot	High	Weak	Yes
Overcast	Warm	Normal	Weak	No

Algorithm

1. Initialise h to the most specific hypothesis in H: <0, 0, ..., 0>.
2. For each training example x:
   - If x is a negative example, ignore it.
   - If x is a positive example, then for each attribute constraint a(i) in h: if a(i) is satisfied by x, do nothing; otherwise replace a(i) by the next more general constraint that is satisfied by x ("?").
3. Output the hypothesis h.

Step-by-step Procedure

1. Import the csv module and store the training data in weather.csv.

2. Open the file, read it with csv.reader() and store the header and the rows in a list.
3. Take the number of attributes n = number of columns − 1 and initialise the hypothesis with "0".
4. For every row, separate the example (first n values) and the label (last value).
5. For a positive example, fill the "0" constraints with the example values and replace every constraint that differs by "?".
6. Print the hypothesis after every example together with its label and display the final hypothesis.

Formulae

h = < a1, a2, ..., an >, ai ∈ { specific value, ?, 0 }

Initial hypothesis: h0 = < 0, 0, ..., 0 >

For a positive example x: ai = ai if ai = xi ; ai = ? otherwise

h satisfies x ⇔ for every i, ai = ? or ai = xi

Python Program

ex-1b.py - D:/ex-1b.py (3.10.0)

File Edit Format Run Options Window Help

```
# FIND-S Algorithm - Weather data (using the csv module)
import csv

rows = []
with open("weather.csv") as f:
    reader = csv.reader(f)
    header = next(reader)
    for r in reader:
        rows.append(r)

print("FIND-S ALGORITHM")
print("----------------")
print(header)
for r in rows:
    print(r)
print()

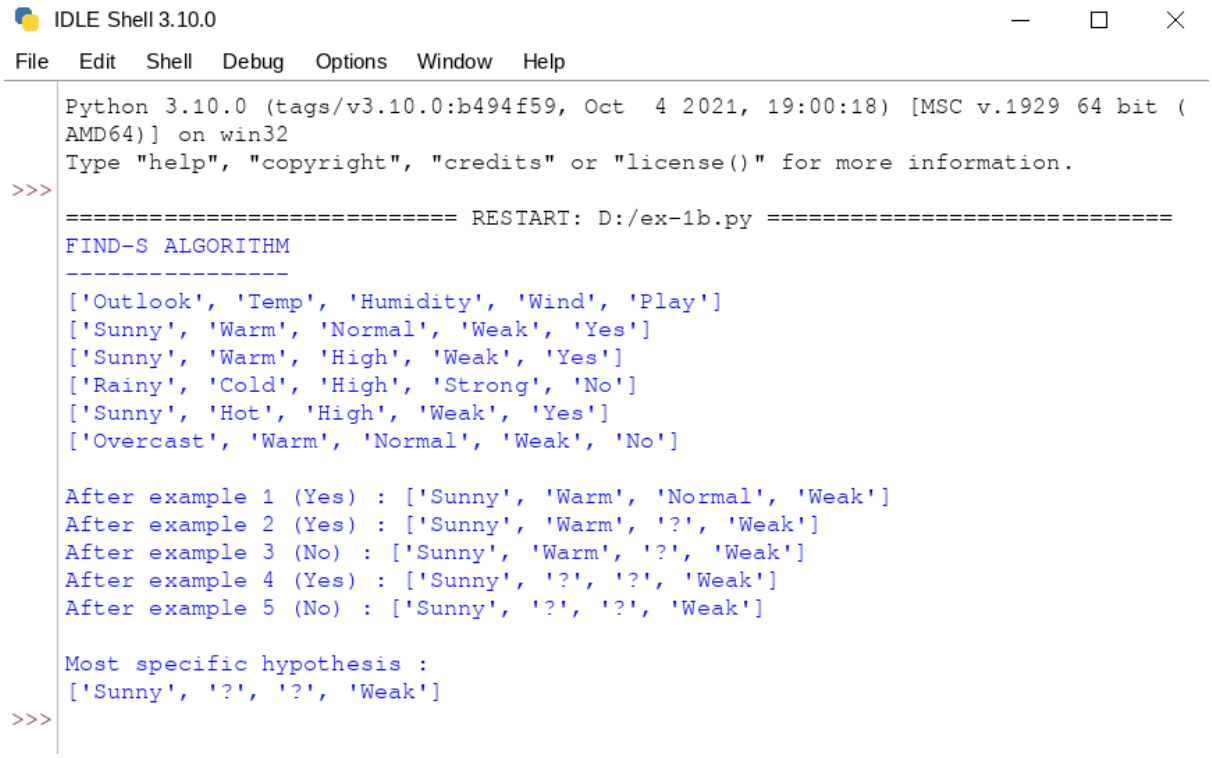
n = len(header) - 1
hypothesis = ["0"] * n

for i in range(len(rows)):
    example = rows[i][:n]
    label = rows[i][n]

    # Generalise the hypothesis only for positive examples
    if label == "Yes":
        for j in range(n):
            if hypothesis[j] == "0":
                hypothesis[j] = example[j]
            elif hypothesis[j] != example[j]:
                hypothesis[j] = "?"
    print("After example", i + 1, "(" + label + ") :", hypothesis)

print()
print("Most specific hypothesis :")
print(hypothesis)
```

Output



Result / Conclusion

The FIND-S algorithm was implemented successfully on the Weather data set using the csv module. The most specific hypothesis is <Sunny, ?, ?, Weak>, i.e. the game is played when the outlook is sunny and the wind is weak. The negative examples 3 and 5 did not change the hypothesis.

Experiment No. 1(c) Date:

Experiment Question

Implement and demonstrate the FIND-S algorithm for finding the most specific hypothesis based on a given set of training data samples. [Sub-program (c): Mango Quality data set, using numpy]

Aim

To implement the FIND-S algorithm in Python using numpy and to find the most specific hypothesis consistent with all the positive training examples of the Mango Quality data set.

Concept

FIND-S is a concept learning algorithm. It starts with the most specific hypothesis in the hypothesis space H and generalises it only as much as is needed to cover each positive training example. Negative examples are ignored.

A hypothesis is a vector of attribute constraints. Each constraint is a specific value, "?" (any value is acceptable) or "0" (no value is acceptable). The output is the maximally specific hypothesis that is consistent with the positive examples, provided the data is free of errors and the target concept is contained in H.

Dataset Description

The Mango Quality data set (mango.csv) has 5 training examples, 4 input attributes (Colour, Size, Smell, Firmness) and the target attribute GoodToBuy (Yes / No).

Colour	Size	Smell	Firmness	GoodToBuy
Yellow	Large	Sweet	Soft	Yes
Yellow	Medium	Sweet	Soft	Yes
Green	Large	Sour	Hard	No
Yellow	Large	Sweet	Firm	Yes
Green	Medium	Sweet	Soft	No

Algorithm

1. Initialise h to the most specific hypothesis in H: <0, 0, ..., 0>.
2. For each training example x:
   - If x is a negative example, ignore it.
   - If x is a positive example, then for each attribute constraint a(i) in h: if a(i) is satisfied by x, do nothing; otherwise replace a(i) by the next more general constraint that is satisfied by x ("?").
3. Output the hypothesis h.

Step-by-step Procedure

1. Import numpy and store the training data in mango.csv.
2. Load the file using np.loadtxt() as a string array and separate the attributes and the target.

3. Find the first positive example using np.where() and take it as the initial hypothesis.
4. For every positive example, keep the values of the hypothesis that match the example and replace the others by "?" using np.where().
5. Print the hypothesis after every example.
6. Display the final most specific hypothesis.

Formulae

h = < a1, a2, ..., an >, ai ∈ { specific value, ?, 0 }

Initial hypothesis: h0 = < 0, 0, ..., 0 >

For a positive example x: ai = ai if ai = xi ; ai = ? otherwise

h satisfies x ⇔ for every i, ai = ? or ai = xi

Python Program

ex-1c.py - D:/ex-1c.py (3.10.0)

File Edit Format Run Options Window Help

```
# FIND-S Algorithm - Mango Quality data (using numpy)
import numpy as np

data = np.loadtxt("mango.csv", delimiter=",", dtype=str, skiprows=1)

print("FIND-S ALGORITHM")
print("----------------")
print(data)
print()

attributes = data[:, :-1]
target = data[:, -1]

# Start with the first positive example
first = np.where(target == "Yes")[0][0]
hypothesis = attributes[first].copy()

for i in range(len(target)):
    if target[i] == "Yes":
        # keep the value if it matches, otherwise replace it by ?
        hypothesis = np.where(hypothesis == attributes[i], hypothesis, "?")
    print("After example", i + 1, ":", hypothesis.tolist())

print()
print("Most specific hypothesis :")
print(hypothesis.tolist())
```

Output



Result / Conclusion

The FIND-S algorithm was implemented successfully on the Mango Quality data set using numpy. The most specific hypothesis is <Yellow, ?, Sweet, ?>. So a mango is good to buy when its colour is yellow and its smell is sweet, whatever the size and the firmness.

Experiment No. 1(d) Date:

Experiment Question

Implement and demonstrate the FIND-S algorithm for finding the most specific hypothesis based on a given set of training data samples. [Sub-program (d): Loan Approval data set, using a function and Python lists]

Aim

To implement the FIND-S algorithm in Python using a function and Python lists and to find the most specific hypothesis consistent with all the positive training examples of the Loan Approval data set.

Concept

FIND-S is a concept learning algorithm. It starts with the most specific hypothesis in the hypothesis space H and generalises it only as much as is needed to cover each positive training example. Negative examples are ignored.

A hypothesis is a vector of attribute constraints. Each constraint is a specific value, "?" (any value is acceptable) or "0" (no value is acceptable). The output is the maximally specific hypothesis that is consistent with the positive examples, provided the data is free of errors and the target concept is contained in H.

Dataset Description

The Loan Approval data set has 5 training examples, 4 input attributes (Income, Credit, Employment, Property) and the target attribute Approved (Yes / No). The data is stored directly in a Python list in the program.

Income	Credit	Employment	Property	Approved
High	Good	Salaried	Owned	Yes
High	Good	Salaried	Rented	Yes
Low	Poor	Self-employed	Rented	No
High	Good	Self-employed	Owned	Yes
Low	Good	Salaried	Owned	No

Algorithm

1. Initialise h to the most specific hypothesis in H: <0, 0, ..., 0>.
2. For each training example x:
   - If x is a negative example, ignore it.
   - If x is a positive example, then for each attribute constraint a(i) in h: if a(i) is satisfied by x, do nothing; otherwise replace a(i) by the next more general constraint that is satisfied by x ("?").
3. Output the hypothesis h.

Step-by-step Procedure

1. Store the training examples in a Python list (Income, Credit, Employment, Property, Approved).
2. Define a function find_s(data) that initialises the hypothesis with "0".
3. Inside the function, loop through the examples and process only the positive examples as in the algorithm.
4. Print the hypothesis after every example.
5. Return the final hypothesis from the function.
6. Call the function with the training data and display the result.

Formulae

h = < a1, a2, ..., an >, ai ∈ { specific value, ?, 0 }

Initial hypothesis: h0 = < 0, 0, ..., 0 >

For a positive example x: ai = ai if ai = xi ; ai = ? otherwise

h satisfies x ⇔ for every i, ai = ? or ai = xi

Python Program

ex-1d.py - D:/ex-1d.py (3.10.0)

File Edit Format Run Options Window Help

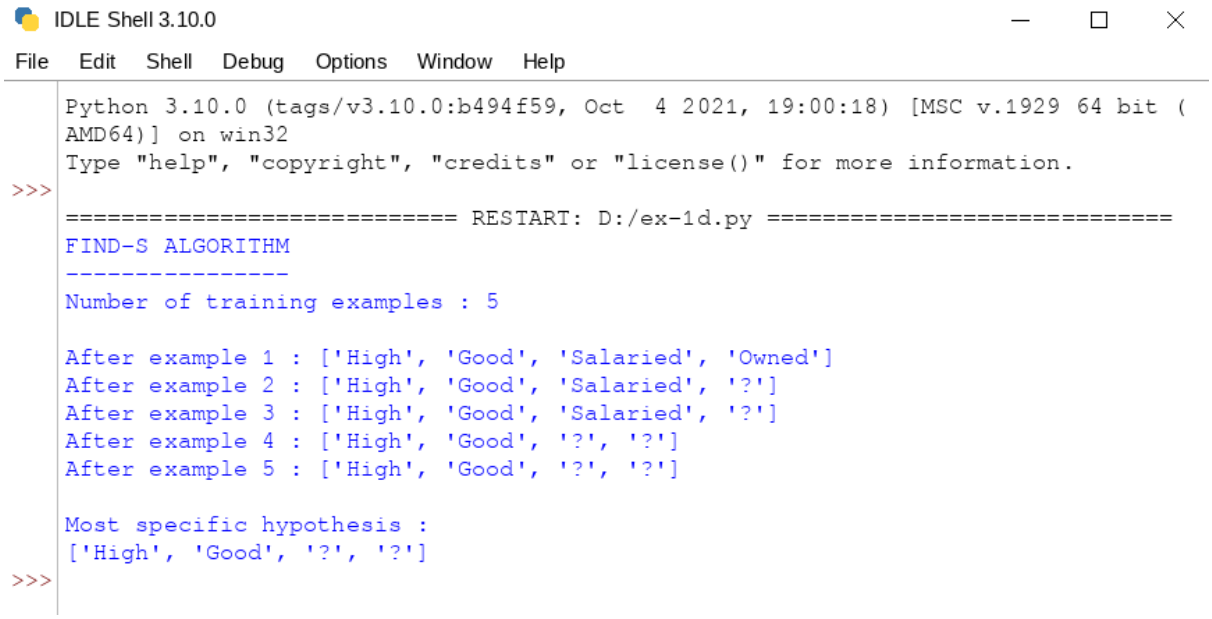
```
# FIND-S Algorithm - Loan Approval data (function and lists)

# Income, Credit, Employment, Property, Approved
training_data = [
    ["High", "Good", "Salaried", "Owned", "Yes"],
    ["High", "Good", "Salaried", "Rented", "Yes"],
    ["Low", "Poor", "Self-employed", "Rented", "No"],
    ["High", "Good", "Self-employed", "Owned", "Yes"],
    ["Low", "Good", "Salaried", "Owned", "No"]
]

def find_s(data):
    n = len(data[0]) - 1
    h = ["0"] * n
    for k in range(len(data)):
        row = data[k]
        if row[-1] == "Yes":
            for j in range(n):
                if h[j] == "0":
                    h[j] = row[j]
                elif h[j] != row[j]:
                    h[j] = "?"
        print("After example", k + 1, ":", h)
    return h

print("FIND-S ALGORITHM")
print("----------------")
print("Number of training examples :", len(training_data))
print()
result = find_s(training_data)
print()
print("Most specific hypothesis :")
print(result)
```

Output



Result / Conclusion

The FIND-S algorithm was implemented successfully on the Loan Approval data set using a function. The most specific hypothesis is <High, Good, ?, ?>, i.e. a loan is approved when the income is high and the credit is good.

Experiment No. 1(e) Date:

Experiment Question

Implement and demonstrate the FIND-S algorithm for finding the most specific hypothesis based on a given set of training data samples. [Sub-program (e): Flu Diagnosis data set, using a class]

Aim

To implement the FIND-S algorithm in Python as a class and to find the most specific hypothesis consistent with the positive examples of the Flu Diagnosis data set, and then to use it to classify new patients.

Concept

FIND-S is a concept learning algorithm. It starts with the most specific hypothesis in the hypothesis space H and generalises it only as much as is needed to cover each positive training example. Negative examples are ignored.

A hypothesis is a vector of attribute constraints. Each constraint is a specific value, "?" (any value is acceptable) or "0" (no value is acceptable). The output is the maximally specific hypothesis that is consistent with the positive examples, provided the data is free of errors and the target concept is contained in H.

Dataset Description

The Flu Diagnosis data set (flu.csv) has 5 training examples, 4 input attributes (Fever, Cough, Headache, BodyPain) and the target attribute Flu (Yes / No).

Fever	Cough	Headache	BodyPain	Flu
High	Yes	Severe	Yes	Yes
High	Yes	Mild	Yes	Yes
Normal	No	Mild	No	No
High	Yes	Severe	No	Yes
Normal	Yes	Mild	Yes	No

Algorithm

1. Initialise h to the most specific hypothesis in H: <0, 0, ..., 0>.
2. For each training example x:
   - If x is a negative example, ignore it.
   - If x is a positive example, then for each attribute constraint a(i) in h: if a(i) is satisfied by x, do nothing; otherwise replace a(i) by the next more general constraint that is satisfied by x ("?").
3. Output the hypothesis h.

Step-by-step Procedure

1. Define a class FindS with a hypothesis h initialised with "0".

2. Add the method learn(x) that generalises h using a positive example x.
3. Add the method predict(x) that returns Yes if the new sample satisfies h, otherwise No.
4. Read flu.csv with pandas and create the lists X (attributes) and y (target).
5. Train the model using only the positive examples and print h after every example.
6. Test the hypothesis on new patients using predict().

Formulae

h = < a1, a2, ..., an >, ai ∈ { specific value, ?, 0 }

Initial hypothesis: h0 = < 0, 0, ..., 0 >

For a positive example x: ai = ai if ai = xi ; ai = ? otherwise

h satisfies x ⇔ for every i, ai = ? or ai = xi

Python Program

ex-1e.py - D:/ex-1e.py (3.10.0)

File Edit Format Run Options Window Help

```
# FIND-S Algorithm - Flu Diagnosis data (using a class)
import pandas as pd

class FindS:
    def __init__(self, n):
        self.h = ["0"] * n

    def learn(self, x):
        for j in range(len(self.h)):
            if self.h[j] == "0":
                self.h[j] = x[j]
            elif self.h[j] != x[j]:
                self.h[j] = "?"

    def predict(self, x):
        for j in range(len(self.h)):
            if self.h[j] != "?" and self.h[j] != x[j]:
                return "No"
        return "Yes"

data = pd.read_csv("flu.csv")
X = data.iloc[:, :-1].values.tolist()
y = data.iloc[:, -1].values.tolist()

model = FindS(len(X[0]))
```

ex-1e.py - D:/ex-1e.py (3.10.0)

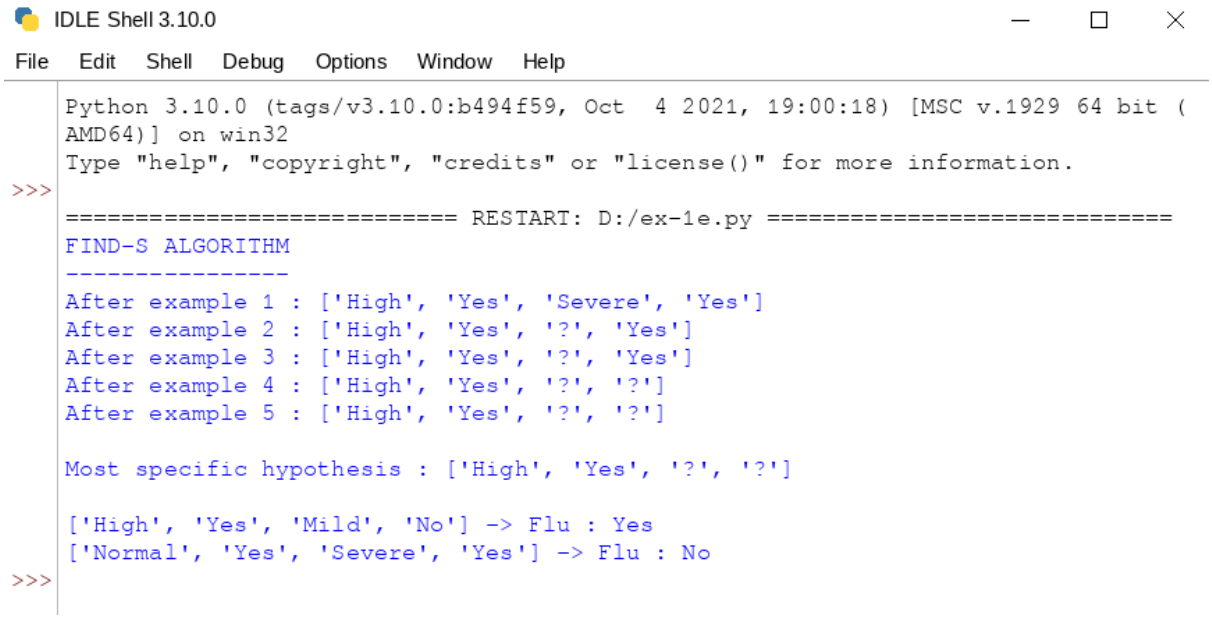
File Edit Format Run Options Window Help

```
print("FIND-S ALGORITHM")
print("----------------")
for i in range(len(X)):
    if y[i] == "Yes":
        model.learn(X[i])
    print("After example", i + 1, ":", model.h)

print()
print("Most specific hypothesis :", model.h)

# Test the hypothesis on new patients
tests = [["High", "Yes", "Mild", "No"],
         ["Normal", "Yes", "Severe", "Yes"]]
print()
for t in tests:
    print(t, "-> Flu :", model.predict(t))
```

Output



Result / Conclusion

The FIND-S algorithm was implemented successfully on the Flu Diagnosis data set using a class. The most specific hypothesis is <High, Yes, ?, ?>, i.e. a patient with high fever and cough has flu. On testing, the patient (High, Yes, Mild, No) was predicted as Flu = Yes and the patient (Normal, Yes, Severe, Yes) as Flu = No.